

Raport diagnostyczny - przechwycenie zapytania SQL

Dokument przedstawia analize zapytania SQL (T-SQL) generowanego przez Entity Framework Core dla wybranego endpointu API. Zgodnie z dopuszczoną alternatywą dla SQL Server Profiler, do przechwycenia zapytania wykorzystano szczegółowe logowanie SQL w EF Core (mechanizm LogTo).

1. Wybrany endpoint

Endpoint	GET /api/visits/today
Kontroler	VisitsController.GetToday() -> VisitService.GetTodayAsync()
Charakterystyka	JOIN dwóch tabel (Patients, AspNetUsers) + filtrowanie po dacie
Autoryzacja	Role: Admin, Lekarz, Rejestratorka

Endpoint zwraca listę wizyt zaplanowanych na bieżący dzień wraz z danymi pacjenta i lekarza - realistyczny przykład zapytania z łączeniem tabel i filtrowaniem zakresu dat.

2. Konfiguracja logowania SQL (EF Core)

W Program.cs (tryb Development) włączono szczegółowe logowanie poleceń bazodanowych:

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>
{
    options.UseSqlServer(connectionString);
    if (builder.Environment.IsDevelopment())
    {
        options.EnableSensitiveDataLogging();
        options.EnableDetailedErrors();
        options.LogTo(Console.WriteLine,
            [DbLoggerCategory.Database.Command.Name], LogLevel.Information);
    }
});
```

3. Wywołanie endpointu i log EF Core

Po uruchomieniu aplikacji lokalnie i wywołaniu endpointu (Swagger: GET /api/visits/today) EF Core wypisuje w konsoli wykonane polecenie wraz z parametrami:

```
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (4ms) [Parameters=[@today='2026-06-15T00:00:00',
      @tomorrow='2026-06-16T00:00:00'], CommandType='Text']
      SELECT [v].[Id], ... FROM [Visits] AS [v]
      INNER JOIN ( ... ) AS [p0] ON [v].[PatientId] = [p0].[Id]
      INNER JOIN [AspNetUsers] AS [a] ON [v].[DoctorId] = [a].[Id]
      WHERE [v].[VisitDate] >= @today AND [v].[VisitDate] < @tomorrow
      ORDER BY [v].[VisitDate]
```

4. Wyizolowane zapytanie T-SQL

Pelne, sparametryzowane zapytanie wygenerowane przez EF Core dla tego endpointu:

```
DECLARE @today datetime2 = '2026-06-15T00:00:00';
DECLARE @tomorrow datetime2 = '2026-06-16T00:00:00';

SELECT [v].[Id], [v].[Description], [v].[DoctorId], [v].[PatientId],
       [v].[Status], [v].[VisitDate],
       [p0].[Id], [p0].[FirstName], [p0].[LastName], [p0].[Pesel], ... ,
       [a].[Id], [a].[UserName], [a].[Email], ...
FROM [Visits] AS [v]
INNER JOIN (
    SELECT [p].[Id], [p].[FirstName], [p].[LastName], [p].[Pesel], ...
    FROM [Patients] AS [p]
    WHERE [p].[IsDeleted] = CAST(0 AS bit)
) AS [p0] ON [v].[PatientId] = [p0].[Id]
INNER JOIN [AspNetUsers] AS [a] ON [v].[DoctorId] = [a].[Id]
WHERE [v].[VisitDate] >= @today AND [v].[VisitDate] < @tomorrow
ORDER BY [v].[VisitDate]
```

5. Opis działania zapytania

- INNER JOIN do tabeli Patients: pobiera dane pacjenta. Zawiera warunek WHERE [p].[IsDeleted] = 0 - to globalny filtr zapytan (soft delete, RODO) automatycznie dodawany przez EF Core.
- INNER JOIN do tabeli AspNetUsers (alias [a]): pobiera dane przypisanego lekarza po DoctorId.
- WHERE [v].[VisitDate] >= @today AND < @tomorrow: filtruje wizyty z bieżącego dnia. Wartości przekazane jako parametry (@today, @tomorrow), a nie wstrzyknięte do tekstu - ochrona przed SQL Injection i możliwość cache'owania planu wykonania.
- ORDER BY [v].[VisitDate]: sortowanie chronologiczne wyników.

6. Komentarz - poprawność

Zapytanie jest poprawne i zgodne z logiką biznesową endpointu: łączy wizyty z pacjentem i lekarzem, ogranicza wynik do jednego dnia i sortuje rosnąco. EF Core wygenerował pojedyncze, sparametryzowane zapytanie (brak problemu N+1 dzięki Include). Filtr daty operuje na kolumnie VisitDate, dla której istnieje indeks nieklastrowany (IX_Visits_VisitDate), co zapewnia efektywne wyszukiwanie zakresu.